

Workers: State via the Tensor

Typed Agentic AI — Building Agentic Businesses

Kodamai

Do it Kodamai Style! (Note 3 in the containers series.)

An agent that reads a request and returns a response is a *container morphism*; that story was told in the first two notes. But real agents are rarely memoryless. A router remembers which tenant it is serving; a ledger assistant carries a running balance; an orchestrator threads a session across many calls. These are *stateful* agents — *workers*, in our vocabulary — and the striking thing is that state needs no new machinery at all. A worker is a container morphism whose *domain has been tensored with one fixed, very simple container*: the *state object*. Unfolding that single tensor produces, automatically and by type-checking alone, exactly the forward/backward pair a stateful process must expose — a way to read the state and produce an answer, and a way to write a new state back. This note pins that down.

1 Containers, morphisms, the tensor

We recall just enough notation from the earlier notes; a reader who has met containers can skim.

A container

A **container** is a pair $S \triangleright P$ of a set S of *shapes* (prompts, events, states) and a family $P : S \rightarrow \mathbf{Set}$ of *positions* (responses, continuations) — for each shape s , a set $P s$ of the positions available at s . Its **extension** is the functor

$$\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} X^{P s} = \sum_{s:S} (P s \rightarrow X).$$

A warning about $\triangleright$. We write $S \triangleright P$ for the *constructor* that pairs a shape set with a position family. This is a piece of syntax, not an operator; in particular it is *not* the composition product. Sequential composition/substitution — the functor-composition product on containers — we write $\triangleleft$, as in the second note. (Some of the internal proof notes write composition as $\triangleright$; here $\triangleright$ is reserved for the constructor, and $\triangleleft$ is composition. §6 depends on keeping the two apart.)

A container morphism

A morphism $(S \triangleright P) \rightarrow (T \triangleright Q)$ is a pair

$$f : S \rightarrow T, \quad f^\# : \prod_{s:S} (Q(f s) \rightarrow P s),$$

a map *forward* on shapes and, for each shape s , a map *backward* on positions. Control flows down (delegate the shape); data flows back up (amalgamate the positions). The backward leg is contravariant, and that single fact governs everything below.

The tensor $\otimes$

The **tensor** (Dirichlet tensor, Day convolution of $\times$) multiplies shapes and *multiplies* positions:

$$(S_0 \triangleright P_0) \otimes (S_1 \triangleright P_1) = (S_0 \times S_1) \triangleright (\lambda(s_0, s_1). P_0 s_0 \times P_1 s_1), \quad \text{unit } y = (1 \triangleright \lambda_. 1).$$

On extensions $\llbracket p \otimes q \rrbracket \cong \llbracket p \rrbracket \otimes \llbracket q \rrbracket$: it is the “ask both, answer both” operator that models parallel, non-interfering interaction.

2 The worker container is the state object

Everything hinges on one container. Fix a set S of states.

The state object

The **state object** on S is the container whose shape set is S and whose position set at *every* shape is S again — a constant position family:

$$\Delta S := S \triangleright (\lambda_. S).$$

This is exactly Robin’s proposed “worker container” $A = S \triangleright (\lambda_. S)$; we shall write ΔS for it throughout, matching the earlier proof notes. Its extension is the **store (costate) comonad**

$$\llbracket \Delta S \rrbracket(X) = \sum_{s:S} X^S = S \times X^S = \text{Store}_S X,$$

a *current state* paired with a *reader* $X^S = (S \rightarrow X)$; and, as a directed container, ΔS is the codiscrete category on S (one arrow between every ordered pair of states). Both facts are established in the companion notes and we lean on them in §5. For now we need only the raw shape/position data.

3 The unfolding theorem

We now confirm — rigorously, step by step — Robin’s guess: that a morphism $\Delta S \otimes X \rightarrow Y$ is a stateful forward/backward pair, with the tensor doing all the work.

Fix an input container $X = X_0 \triangleright X_1$ and an output container $Y = Y_0 \triangleright Y_1$.

Lemma 1 (The tensored domain). $\Delta S \otimes X = (S \times X_0) \triangleright (\lambda(s, x_0). S \times X_1 x_0)$.

Proof. Apply the tensor formula to $\Delta S = S \triangleright (\lambda_. S)$ and $X = X_0 \triangleright X_1$. Shapes multiply: $S \times X_0$. Positions multiply: at a shape (s, x_0) the position set is $(\lambda_. S)(s) \times X_1 x_0 = S \times X_1 x_0$. $\square$ $\square$

Read the tensored domain aloud: a shape is a *current state together with an input shape*, and a position is a *state together with an input position*. The constant S -fibre of ΔS has spread a copy of S across every position of X — and it did so *by multiplication*, precisely because the tensor multiplies positions.

Theorem 1 (Unfolding a worker). *A container morphism $w : \Delta S \otimes X \rightarrow Y$ is exactly a pair*

$$\text{fwd} : S \times X_0 \longrightarrow Y_0, \quad \text{bwd} : \prod_{(s, x_0)} (Y_1(\text{fwd}(s, x_0)) \longrightarrow S \times X_1 x_0).$$

Equivalently, bwd splits by the universal property of $\times$ into two independent components

$$\begin{aligned} \text{bwd}_S : \prod_{(s, x_0)} Y_1(\text{fwd}(s, x_0)) &\rightarrow S && \text{(the writeback of a new state),} \\ \text{bwd}_X : \prod_{(s, x_0)} Y_1(\text{fwd}(s, x_0)) &\rightarrow X_1 x_0 && \text{(the ordinary backward map on } X\text{).} \end{aligned}$$

Proof. By the definition of a container morphism, $w : \Delta S \otimes X \rightarrow Y$ is a pair $(f, f^\#)$ with f forward on the shapes of $\Delta S \otimes X$ and $f^\#$ backward on its positions. By Lemma 1 the shape set of the domain is $S \times X_0$ and the position set at (s, x_0) is $S \times X_1 x_0$, while $Y = Y_0 \triangleright Y_1$ has shapes Y_0 and positions Y_1 . Hence

$$f : S \times X_0 \rightarrow Y_0 \quad \text{and} \quad f^\# : \prod_{(s, x_0)} (Y_1(f(s, x_0)) \rightarrow S \times X_1 x_0),$$

which are fwd and bwd verbatim. Finally, a map into a product $S \times X_1 x_0$ is, uniquely, a pair of maps into S and into $X_1 x_0$; this is the split $\text{bwd} = \langle \text{bwd}_S, \text{bwd}_X \rangle$. $\square$ $\square$

THE DEEPER POINT

State is not an extra feature of a worker; it is *one fixed tensor factor*. Tensoring the domain with ΔS threads a state S through the interface, and the container-morphism discipline then *forces* the two things a stateful process must supply: a forward map that reads (state, input) and returns an output, and a backward map that, from each output position, both *writes back* a new state and answers the original input. Neither was postulated — both are consequences of the types. This is “correct by construction” for stateful agents: the state discipline is checked, not hoped for.

Corollary 1 (Robin’s guess is confirmed). *With $A = \Delta S = S \triangleright (\lambda \dots S)$ and the tensor multiplying positions, a morphism $A \otimes X \rightarrow Y$ unfolds to precisely*

$$\text{fwd} : S \times X_0 \rightarrow Y_0, \quad \text{bwd} : (s, x) \mapsto (Y_1(\text{fwd } s \, x) \rightarrow S \times X_1 x),$$

Robin’s stated forward/backward pair, with no correction needed. The only clarification is book-keeping: the x in bwd is the input shape $x_0 : X_0$, and bwd is genuinely dependent (its domain $Y_1(\text{fwd}(s, x_0))$ varies with the output shape it lands on).

This morphism is exactly the **Worker** of the companion note (**delta-state-object-and-workers**): a $\Delta S \otimes p \rightarrow q$ is there unpacked as a forward $f : S \times A \rightarrow C$ together with a backward split into $f_1^\#$ (writeback into S) and $f_2^\#$ (back-map into Ba). Theorem 1 is that unpacking, with $p = X$ and $q = Y$; the present note simply isolates it as the definition of “state via the tensor” and verifies each step. Under this reading a **worker of state S from X to Y** is an element of $\text{Cont}(\Delta S \otimes X, Y)$, and workers compose with the state *multiplying*: a state- S worker followed by a state- T worker is a state- $(S \times T)$ worker, because $\Delta S \otimes \Delta T = \Delta(S \times T)$ (Lemma 2 below). That is the whole reason the tensor — and not some other product — is the right home for state.

4 An example: a running balance

Let the state be an integer balance, $S = \mathbb{Z}$. Take the input interface X to be “deposit or query”: $X_0 = \{\text{dep } n \mid n \in \mathbb{N}\} \cup \{\text{bal}\}$, with positions $X_1(\text{dep } n) = \{*\}$ (a deposit just needs an acknowledgement) and $X_1(\text{bal}) = \{*\}$. Let the output interface be $Y = \mathbb{Z} \triangleright (\lambda \dots \{*\})$: an output shape is the number reported, with a single trivial position.

A worker $w : \Delta S \otimes X \rightarrow Y$ is, by Theorem 1:

$$\text{fwd}(s, \text{dep } n) = s, \quad \text{fwd}(s, \text{bal}) = s,$$

(report the balance, before or after is a design choice — here we report the pre-state), and, at each shape, a backward map $\text{bwd} : \{*\} \rightarrow S \times X_1$ giving the new state and the input acknowledgement:

$$\text{bwd}_S(s, \text{dep } n)(*) = s + n, \quad \text{bwd}_S(s, \text{bal})(*) = s, \quad \text{bwd}_X(\dots)(*) = *.$$

The forward map *reads* the balance to answer; the backward map *writes* the updated balance $s + n$ on a deposit and leaves it untouched on a query. The type of w guaranteed, before any run, that exactly these two pieces — a read-and-answer and a state-writeback — had to be supplied, and nothing else could sneak in. Compose two such workers and the state set becomes $\mathbb{Z} \times \mathbb{Z}$, one register per stage, exactly as §5 predicts.

5 The store comonad and the lens view

The unfolding of §3 is not an isolated trick; it is the container face of two very familiar objects.

The store comonad. We saw $\llbracket \Delta S \rrbracket(X) = S \times X^S = \text{Store}_S X$. The workers with a *fixed* state S are precisely the coKleisli morphisms of the (graded) comonad $p \mapsto \Delta S \otimes p$: a worker $\Delta S \otimes X \rightarrow Y$ is a coKleisli arrow $X \rightarrow Y$, and worker composition is coKleisli composition. The state object’s own comonad structure — counit “read the current position” and comultiplication “duplicate the register” — is the store comonad of Uustalu–Vene, and ΔS as a directed container is the codiscrete category on S . So “state via the tensor” is the store comonad, presented one level down, at the container rather than the functor.

The lens view. Specialise Theorem 1 to a *monomial* interface with a trivial input shape. Take $X = y$ (so $X_0 = 1$, $X_1 = 1$) and $Y = A \triangleright (\lambda_.B)$. Then a worker $\Delta S \otimes y = \Delta S \rightarrow A \triangleright (\lambda_.B)$ is exactly a pair

$$\text{get} : S \rightarrow A, \quad \text{put} : S \times B \rightarrow S,$$

because $\text{fwd} : S \rightarrow A$ and $\text{bwd}_S : \prod_s (B \rightarrow S)$, i.e. $\text{put} : S \times B \rightarrow S$ (and bwd_X is trivial). This is precisely a **lens** $(S, S) \rightleftarrows (A, B)$: the forward leg is get , the state-writeback is put . A general worker $\Delta S \otimes X \rightarrow Y$ is therefore a *lens with an extra input interface*: the ΔS factor contributes the bidirectional get/put , and the X factor contributes an ordinary container morphism’s delegate/amalgamate. This is the *lens/Para* reading of the companion note: state is a *parameter* that tensors along composition, which is exactly the “parameters tensor” law of categorical cybernetics (Gavranović), the same law that drives learners and open games.

THE DEEPER POINT

One picture, three names. *Downstairs* (containers): a worker is a forward/backward pair with a state-writeback. *One level up* (functors): it is a coKleisli arrow of the store comonad $S \times (-)^S$. *Sideways* (optics): it is a lens, get/put , with an input interface bolted on. The tensor is what makes all three coincide.

6 State via the tensor, not via composition ($\otimes \neq \triangleleft$)

It matters that this is the *tensor*. There is a second, superficially similar way to stack state — *composition* of containers, $p \triangleleft q$, whose extension is functor composition $\llbracket p \rrbracket \circ \llbracket q \rrbracket$ (the note reserves $\triangleleft$ for this; the internal proof notes write it $\triangleright$). The two genuinely differ, and the difference is exactly the difference between *entangling* states and *nesting* them.

Lemma 2 (States multiply under $\otimes$). $\Delta S \otimes \Delta T = \Delta(S \times T)$ on the nose, and $\Delta 1 = y$. Under composition, by contrast, $\Delta S \triangleleft \Delta T$ has shape set $S \times T^S$ (a state plus a branch map $S \rightarrow T$) and is not $\Delta(S \times T)$.

Proof. For $\otimes$: both sides have shape set $S \times T$ and, at every shape, position set $S \times T$; and $\Delta 1 = (1 \triangleright \lambda_.1) = y$. For $\triangleleft$: $\llbracket \Delta S \rrbracket(\llbracket \Delta T \rrbracket(X)) = S \times (T \times X^T)^S = S \times T^S \times X^{S \times T}$, so a shape records a state $s \in S$ and a branch map $g : S \rightarrow T$. $\square$ $\square$

Corollary 2 ($\otimes \neq \triangleleft$, witnessed). At $|S| = |T| = 2$, $\Delta S \otimes \Delta T = 4 \cdot y^4$ while $\Delta S \triangleleft \Delta T = 8 \cdot y^4$. The two products are not equal, and not related by taking a fibre of the other. They are, however, a **retract**: there are container maps $\sigma : \Delta S \otimes \Delta T \rightarrow \Delta S \triangleleft \Delta T$ and r back with $r \circ \sigma = \text{id}$ but $\sigma \circ r \neq \text{id}$; the retracting idempotent collapses each branch map $g : S \rightarrow T$ to the constant at its self-evaluation $g(s)$.

The moral for stateful agents. *State via the tensor* (this note) entangles the two registers into a single product state $S \times T$ — the two workers share nothing but multiply their state spaces. *State via composition* nests a store inside a store (a **store-of-store**), recording not just a state but a whole map $S \rightarrow T$ of how the inner state depends on the outer; that extra data is exactly the branch map g that the retraction throws away. The fwd/bwd unfolding of §3 is a *tensor* phenomenon: the clean split into “writeback into S ” plus “back-map into X_1 ” comes from the tensor’s position-product $S \times X_1 x_0$. Had we nested with $\triangleleft$ instead, a position would carry a branch map and the split would not be this clean — a different, strictly larger, discipline.

THE DEEPER POINT

Two ways to stack state, one carrier ΔS . The tensor multiplies state spaces ($\Delta S \otimes \Delta T = \Delta(S \times T)$, $4y^4$); composition nests stores ($\Delta S \triangleleft \Delta T$, $8y^4$). They are not equal and not fibre-related — they are an explicit retract, and the store comonad’s own comultiplication lives in the extra data that $\triangleleft$ sees and $\otimes$ collapses. “State via the tensor” is the entangling discipline: workers whose composition multiplies the state, cleanly currying past $\otimes$ and giving a genuine stateful function type $[X, Y]_\otimes$.

Summary

A worker is a container morphism out of a tensored domain $\Delta S \otimes X$, where $\Delta S = S \triangleright (\lambda \dots S)$ is the state object. Unfolding the tensor — and nothing more — turns such a morphism into exactly the forward/backward pair

$$\text{fwd} : S \times X_0 \rightarrow Y_0, \quad \text{bwd} : \prod_{(s, x_0)} (Y_1(\text{fwd}(s, x_0)) \rightarrow S \times X_1 x_0),$$

with the backward leg splitting into a state-*writeback* and an ordinary back-map. This is Robin’s guess, confirmed with no correction. It is the store comonad seen at container level, and a lens with an input interface seen sideways; and it is genuinely the *tensor*’s doing — state via $\otimes$ multiplies state spaces, and differs from state via composition $\triangleleft$, which nests them ($4y^4$ vs $8y^4$; a retract, not an equality). State, once again, is not new machinery: it is one tensor factor, and the type checker supplies the rest.

Do it Kodamai Style!